实验 Java4

先把 jar 包下载下来

WpSystem	2024/2/28 12:04	文件夹	
httpRequest4j.iml	2024/3/26 11:00	IML 文件	1
httpRequest4j.jar	2024/4/9 16:51	Executable Jar File	6
Reuqest.class	2024/3/26 17:28	CLASS 文件	4

使用命令行查看包里有什么类，方便我们去通过反射去查看类中的方法

```
G:\>jar tf httpRequest4j.jar
META-INF/MANIFEST.MF
.idea/
.idea/.gitignore
.idea/artifacts/
.idea/artifacts/httpRequest4j_jar.xml
.idea/misc.xml
.idea/modules.xml
.idea/workspace.xml
httpRequest4j.iml
META-INF/
Reuqest.class

G:\>
```

试用水印

看到有一个 Reuqest(应该是 request？)的类
编写代码

```java
import javax.sql.rowset.serial.SerialJavaObject;
import java.io.File;
import java.lang.reflect.Method;
import java.net.URL;
import java.net.URLClassLoader;
public class Main {

    public static void main(String[] args) throws Exception {
      String path="file:G:/httpRequest4j.jar";
      URLClassLoader classload=new URLClassLoader(new URL[]{new URL(path)});

      Class a=classload.loadClass( name: "Reuqest");
        Method[] t = a.getDeclaredMethods();
        for (Method method : t) {
            System.out.println("方法名: " + method.getName());
          Class[] parameterTypes = method.getParameterTypes();
            System.out.print("参数: (");
            for (int i = 0; i < parameterTypes.length; i++) {
                System.out.print(parameterTypes[i].getName());
                if (i < parameterTypes.length - 1) {
                    System.out.print(", ");
                }
            }
            System.out.println(")");
        }
        Object b = a.newInstance();
       /* Method methon = a.getMethod("方法名",主机地址,端口,消息);
        for(int i=0;i<28;i++){
            Object c = methon.invoke(b,10.10.74.158:5000,322081212+i);
        }*/
```

```
进程已结束，退出代码为 0
```

String path="file:G:/httpRequest4j.jar";

URLClassLoader classload=new URLClassLoader(new URL[]{new URL(path)});

将 jar 包加载进程序

Class a=classload.loadClass("Reuqest");

Method[] t = a.getMethods();

获得一个 class 类

并获取它里面的方法

Getmethons 只能获取公有类型的方法

循环打印方法和参数

```
F:\jdk17\bin\java.exe "-javaagent:F:\ij\IntelliJ IDEA Community Edition 2023.3.6\lib\idea_rt.jar=61258:F:\ij\IntelliJ IDEA Community Edition 2023.3.6\bin" -D
方法名: main
参数: ([Ljava.lang.String;)
方法名: wait
参数: (long, int)
方法名: wait
参数: ()
方法名: wait
参数: (long)
方法名: equals
参数: (java.lang.Object)
方法名: toString
参数: ()
方法名: hashCode
参数: ()
方法名: getClass
参数: ()
方法名: notify
参数: ()
方法名: notifyAll
参数: ()

进程已结束，退出代码为 0
```

通过 getDeclaredMethods 方法获取私有的方法

试用水印

```
运行  Main
F:\jdk17\bin\java.exe "-javaagent:F:\ij\IntelliJ IDEA Community Edition 2023.3.6\lib\idea_rt.jar=61996:F:\ij\IntelliJ IDEA Community Edition 2023.3.6\bin" -D
方法名: main
参数: ([Ljava.lang.String;)

进程已结束，退出代码为 0
```

直接运行这个 jar 包

```
G:\>java -jar httpRequest4j.jar
Usage: java -jar Request.jar <server_address:port> <message>

G:\>
```

发现需要两个参数，所以发送消息的方法是一个带有两个参数的方法；（

通过反编译，查看request类的源码

```
10
11 public class Reuqest
12 {
13
14         public Reuqest()
15         {
16         }
17
18         public static void main(String args[])
19         {
20                 String serverAddress;
21                 String message;
22                 if (args.length != 2)
23                 {
24                         System.out.println("Usage: java -jar Request.jar <server_address:port> <message>");
25                         return;
26                 }
27                 serverAddress = args[0];
28                 message = args[1];
29                 HttpURLConnection con;
30                 DataOutputStream out;
31                 Throwable throwable;
32                 Exception exception;
33                 URL url = new URL((new StringBuilder()).append("http://").append(serverAddress).toString());
34                 con = (HttpURLConnection)url.openConnection();
35                 con.setRequestMethod("POST");
36                 con.setRequestProperty("Content-Type", "application/json");
37                 String jsonInputString = message;
38                 con.setDoOutput(true);
39                 out = new DataOutputStream(con.getOutputStream());
40                 throwable = null;
41                 try
42                 {
43                         out.write(jsonInputString.getBytes(StandardCharsets.UTF_8));
44                         out.flush();
45                 }
46                 catch (Throwable throwable2)
47                 {
48                         throwable = throwable2;
49                         throw throwable2;
50                 }
51                 finally
52                 {
53                         if (out == null) goto _L0; else goto _L0
54                 }
55                 if (out != null)
56                         if (throwable != null)
57                                 try
58                                 {
59                                         out.close();
60                                 }
61                                 catch (Throwable throwable1)
62                                 {
63                                         throwable.addSuppressed(throwable1);
64                                 }
65                         else
66
```

将类实例化成对象 b

Object b = a.newInstance();

Method methon = a.getMethod("方法名",主机地址：端口,消息);

最后使用 methon.invoke 方法去调用对象 b 中获得的这个方法

Object c = methon.invoke(b,10.10.74.158:5000,322081212+i);

要想实现全班签到只需要做一个循环去不断调用这个方法并改变消息的值

试用水印